

# 計算 First 集合

コンパイラの構文解析

添付のアルゴリズムで *First* を計算する。

*User context*

*Attachments:*

- *first-[alg.txt](#)*: `/Volumes/home/spool/first-alg.txt`

▶ 2 previous messages

添付ファイルにはFIRST集合の計算アルゴリズムがありましたが、**計算対象の文法（生成規則）が記載されていません。**

計算したい文法を、次のような形式で送ってください。空列は `epsilon` で表します。

```
S : A B ;  
A : a A | epsilon ;  
B : b ;
```

いただいた文法について、添付のアルゴリズムに従って初期化し、要素が増えなくなるまでの計算過程と最終的なFIRST集合を示します。

特定の文法でなく、入力ファイルから読み取って *First* を計算するプログラムを作ってください。

▶ 2 previous messages

Pythonプログラムを作成しました。添付の手順1〜4に従い、集合が変化しなくなるまで反復してFIRSTを計算します。

- プログラム [first.py](#)
- 入力例 [grammar.txt](#)
- 使い方 [README.md](#)

保存先のディレクトリで実行します。

```
python3 first.py grammar.txt
```

右辺の全接尾語のFIRSTも表示する場合：

```
python3 first.py grammar.txt --all
```

入力は `A : a A | epsilon ;` 形式です。左辺に現れる記号を非終端記号、それ以外を終端記号と判定します。yaccの宣言や意味動作には対応していません。

空列・左再帰・相互再帰など6種類の文法と、不正入力4種類で動作確認済みです。

空列の表現は `%empty` と書くことにする。

#### ▶ 2 previous messages

空列の表現を `%empty` に変更しました。計算結果の表示も `%empty` になります。

```
A : a A | %empty ;
```

[プログラム](#)、[入力例](#)、[説明](#)を更新し、動作確認済みです。`epsilon` は通常の記号として扱います。

`-v` をつけて実行すると、アルゴリズムの `1` , `2` , `3` , `4a` , `4b wo`

#### ▶ 2 previous messages

[first.py](#) に `-v` を追加しました。

```
python3 first.py grammar.txt -v
```

手順1・2・3の初期化と、各反復の`4a`・`4b`について、適用理由・更新前後の集合・追加要素を表示します。変化がない場合と終了判定も表示します。

`--all` と併用でき、通常実行と計算結果が一致することを確認済みです。

プログラムはどこにありますか？

プログラムは `first.py` にあります。

保存場所：

```
/Users/shojiyuen/Documents/Codex/2026-09-17/new-chat/outputs/first.py
```

```
~/github/26compiler/src/first-alg ni
```

`~/github/26compiler/src/first-alg` にコピーする。こちらで継続して開発する。

#### ▶ 2 previous messages

コピーはすでにこちらでしている。

承知しました。追加のコピーは行いません。今後は `~/github/26compiler/src/first-alg` を開発対象にします。

なお、先ほどのコピー処理で `first.py` の内容は書き込まれましたが、ファイル属性の設定で停止しました。残りの2ファイルは変更していません。

`-T` というオプションを追加する。横軸にステップ数，縦軸にFIRSTの引数をならべて，内容はFirstの計算結果を記入して表形式で見やすいものにする。

#### ▶ 4 previous messages

`~/github/26compiler/src/first-alg/first.py` に `-T` を追加しました。

```
python3 first.py grammar.txt -T
```

- 横軸：ステップ0（初期化後）、以後 $4a \cdot 4b$ ごと
- 縦軸：FIRSTの全引数
- 各セル：その時点のFIRST集合

横長の表は列を分割して表示します。 `-v` との併用も可能です。計算結果の一致と表の表示を確認済みです。

横軸に 1, 2, 3 も含めて表示する

▶ 2 previous messages

書き込みを承認したので修正してください。

▶ 2 previous messages

開発先の `first.py` を修正しました。

```
python3 first.py grammar.txt -T
```

横軸は 1: init、2: init、3: init、4: 4(a)、5: 4(b) …となります。未初期化は `-`、空集合は `{}` で区別します。動作確認済みです。

`-v` オプションに *First* の定義域を最初にリストで表示する。

▶ 2 previous messages

開発先の `first.py` を修正しました。`-v` を付けると、計算過程の前に定義域  $X = N \cup P0_p$  の要素数と全要素を、重複なしのリストで表示します。

```
python3 first.py grammar.txt -v
```

空列は `%empty` と表示します。動作確認済みです。

`-T` オプションの際に変化があった項目に `!` をつけるようにする。`--csv` を追加して、`csv` 形式でも出力できるようにする。その際には、折り返しは考慮せず、大きな一つの表として作成する。`--csv` 出力ファイル とする。`-T --csv` の併用も可能とする。

▶ 2 previous messages

開発先の `first.py` を修正しました。

- `-T` : 前のステップから変化したセルに `!` を付けます。初期化も含みます。

- `--csv 出力ファイル` : 全ステップを分割しない1つのCSV表として保存します。 `!` も含めます。
- 両方を併用できます。

```
python3 first.py grammar.txt -T --csv first.csv
```

動作確認済みです。

`--csv` で出力するとき、値がまだ全く計算されていない項目には、未定義 という表示とする。

#### ▶ 2 previous messages

修正しました。 `--csv` では、まだ初期化されていないセルを 未定義 と表示します。計算済みの空集合 `{}` と区別できます。

`-T` の表示は従来どおりです。動作確認済みです。

`csv` は 未定義 ではなく、`--` を表示するようにする。

#### ▶ 2 previous messages

修正しました。CSVの未初期化セルは `--` と表示されます。動作確認済みです。